
Lecture 7(1)

Recap

A little bit of everything





**Please ask
questions
if you have
any, that's
the goal!**

Sorry, Homework #1 Has No Correct Answer!

It's a design choice.

That assignment is designed to test your skills and thinking when faced with an impossible problem.

This helps us adjust next assignments based on your knowledge.

Problems with the dataset:

- HEAVILY imbalanced (why?)
- Very noisy labels (why?)
- What it could be good for?



This is not the case for next assignments

Next assignments are going to be: NER, LLM
Finetuning (working on compute for your
assignments) and RAG.



**Any more
questions
about
homework
#1?**

Activation Functions

Do you know what helps you make your point crystal clear? Lists like this one:

- They're simple
- You can organize your ideas clearly
- You'll never forget to buy milk!

And the most important thing: the audience won't miss the point of your presentation

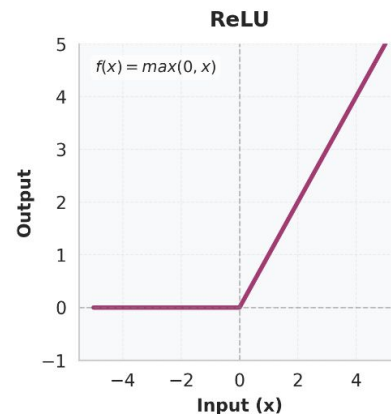
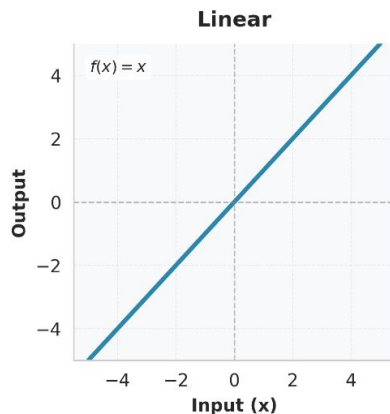


- Linear Activation

Use case: Output layer for regression problems

Why: Allows the network to output continuous values without bounds

Limitation: linear

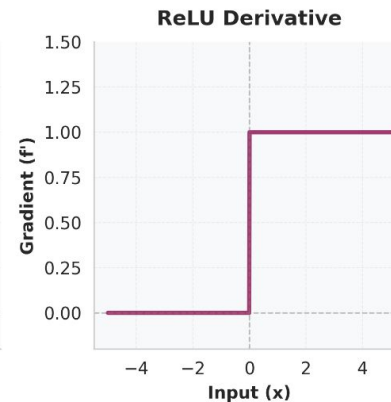
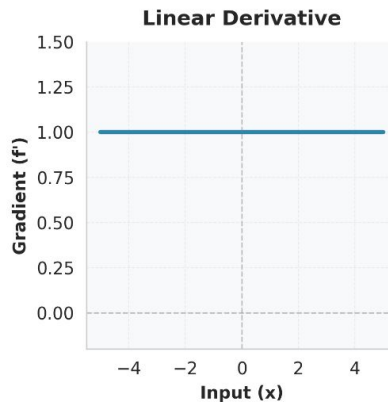


- ReLU (Rectified Linear Unit)

Use case: Hidden layers in most deep neural networks, especially CNNs

Why: Fast to compute, helps avoid vanishing gradients, promotes sparse activation

Limitation: Can suffer from "dying ReLU" problem (neurons that stop learning)



- ELU (Exponential Linear Unit)

Use case: Deep networks where smoother gradients are beneficial

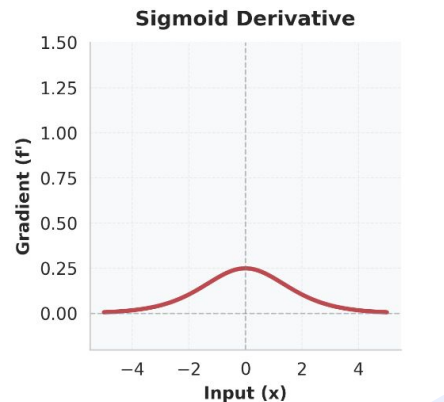
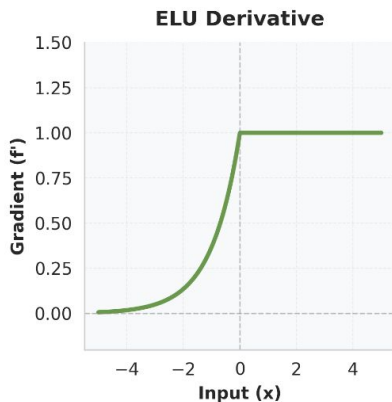
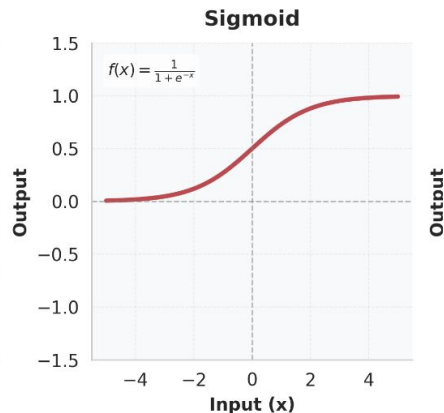
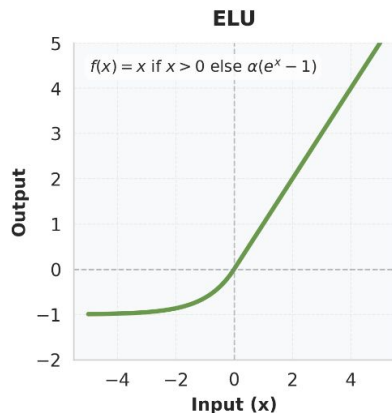
Why: Smoother than ReLU, can produce negative outputs (helps with mean activation closer to zero)

- Sigmoid

Use case: Output layer for binary classification, gates in LSTM/GRU cells

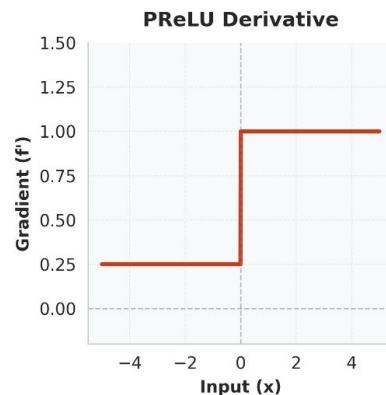
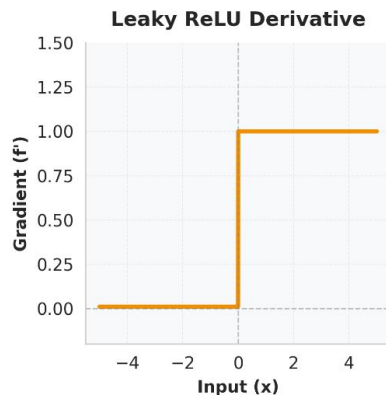
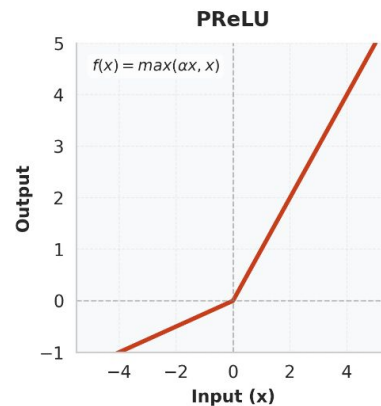
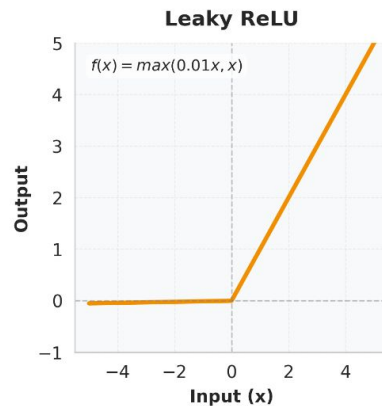
Why: Outputs values between 0 and 1, interpretable as probabilities

Limitation: Suffers from vanishing gradients, rarely used in hidden layers anymore



– Leaky ReLU / PReLU (Parametric ReLU)

Use case: Alternative to ReLU in hidden layers when dying ReLU is a concern
Why: Allows small negative gradients, preventing neurons from completely dying

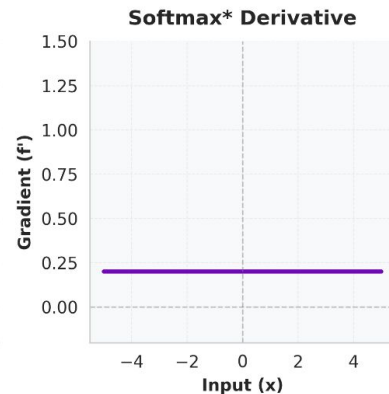
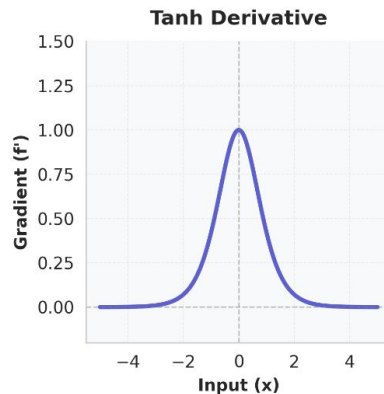
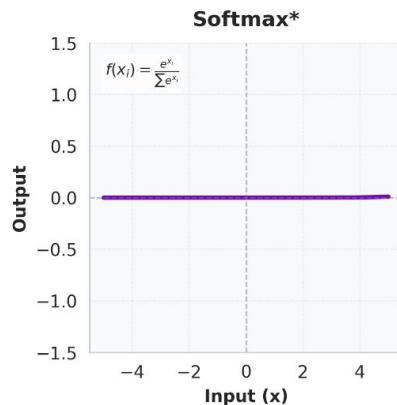
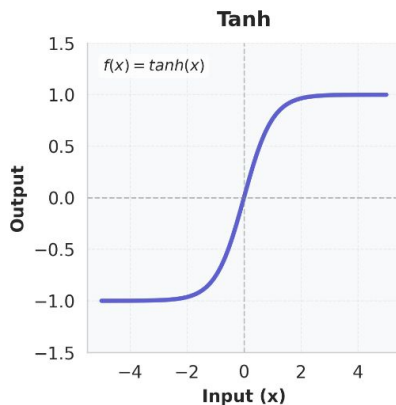


- Tanh (Hyperbolic Tangent)

Use case: Hidden layers in RNNs, situations where zero-centered outputs are preferred

Why: Outputs between -1 and 1, zero-centered (better than sigmoid)

Limitation: Still suffers from vanishing gradients



- Softmax

Use case: Output layer for multi-class classification

Why: Converts logits to probability distribution (outputs sum to 1)

- **Swish / SiLU (Sigmoid Linear Unit)**

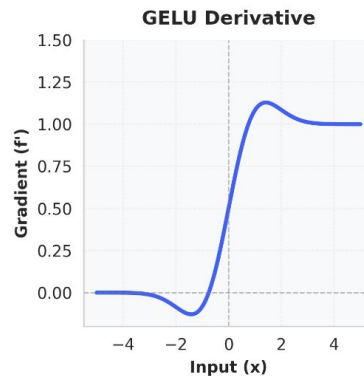
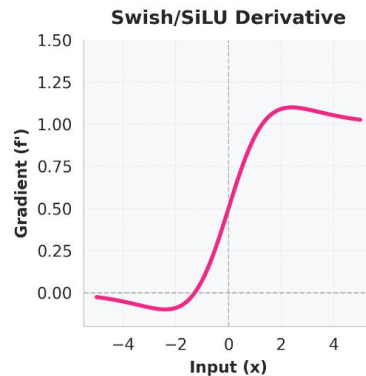
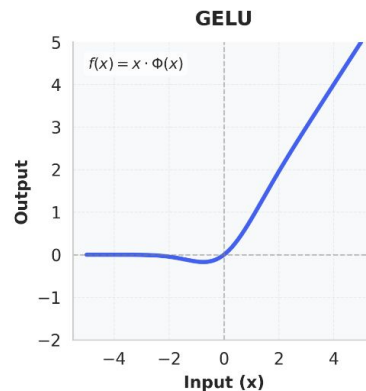
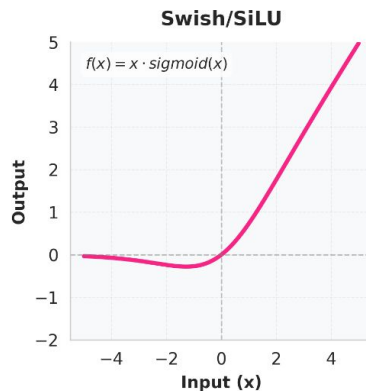
Use case: Modern deep networks, especially when performance is critical

Why: Self-gated, smooth, often outperforms ReLU in deeper networks

- **GELU (Gaussian Error Linear Unit)**

Use case: Transformer models (BERT, GPT), state-of-the-art NLP architectures

Why: Smooth, stochastic regularization properties, empirically strong performance



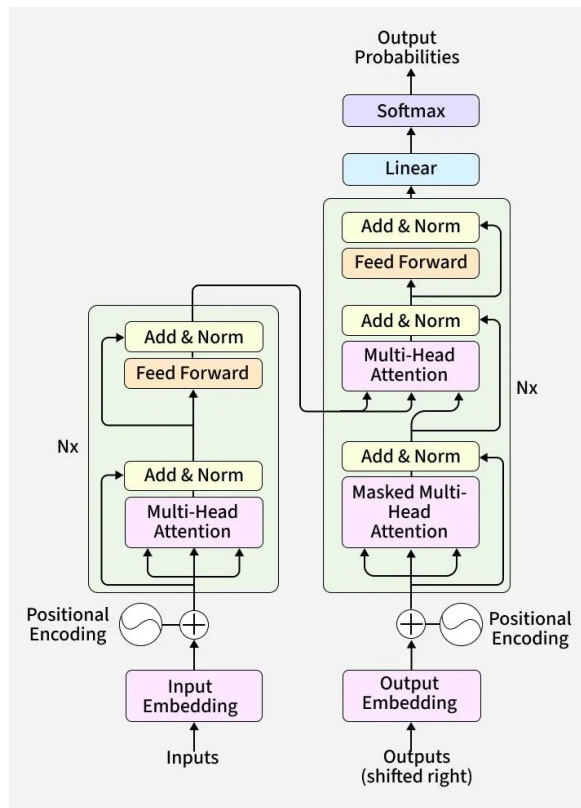


**Are some
topics still
unclear to
you?**



Be honest

Hello again, Transformer





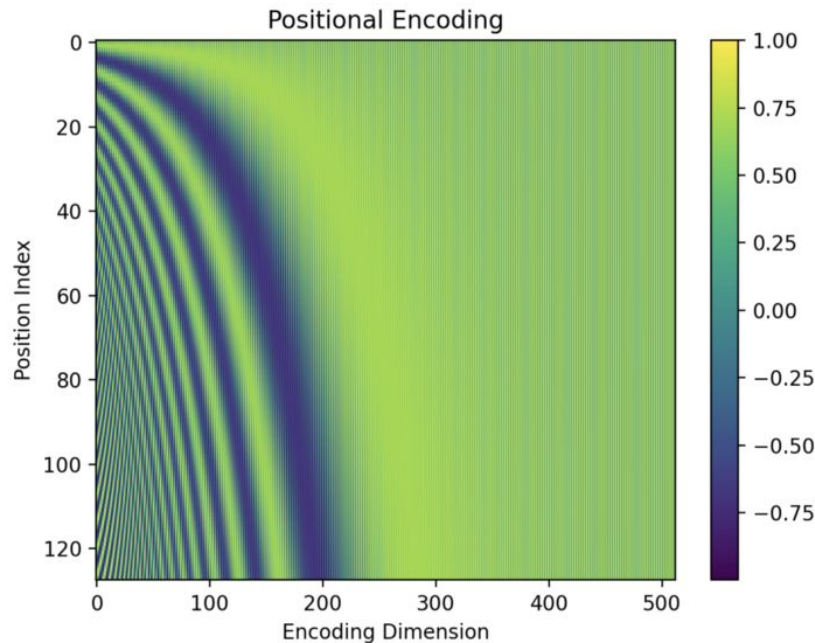
**Why do
we need
transformers?**

Positional Encoding

Unlike RNNs, transformers lack an inherent understanding of word order since they process data in parallel. To solve this problem Positional Encodings are added to token embeddings providing information about the position of each token within a sequence.

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d_{model}})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d_{model}})$$



Self Attention Mechanism

The self attention mechanism allows transformers to determine which words in a sentence are most relevant to each other. This is done using a scaled dot-product attention approach:

Each word in a sequence is mapped to three vectors:

Query (Q)

Key (K)

Value (V)

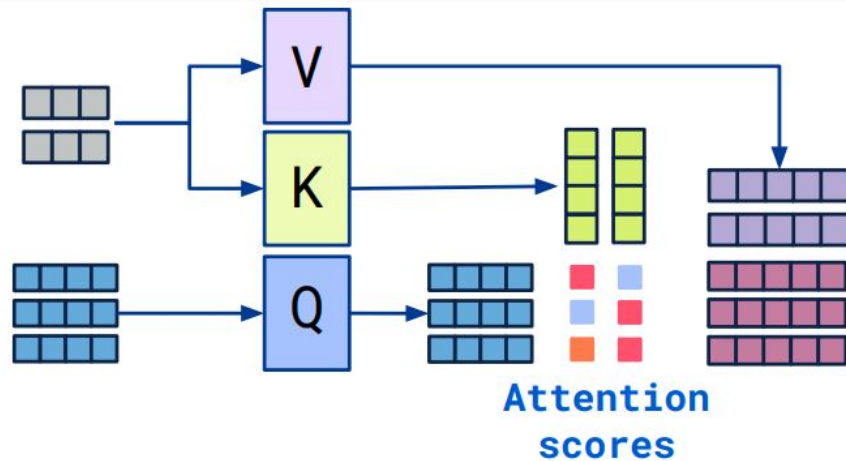
Attention scores are computed as:

$\text{Attention}(Q,K,V) = \text{softmax}(QK^Tdk)V$

$\text{Attention}(Q,K,V) = \text{softmax}(dk$

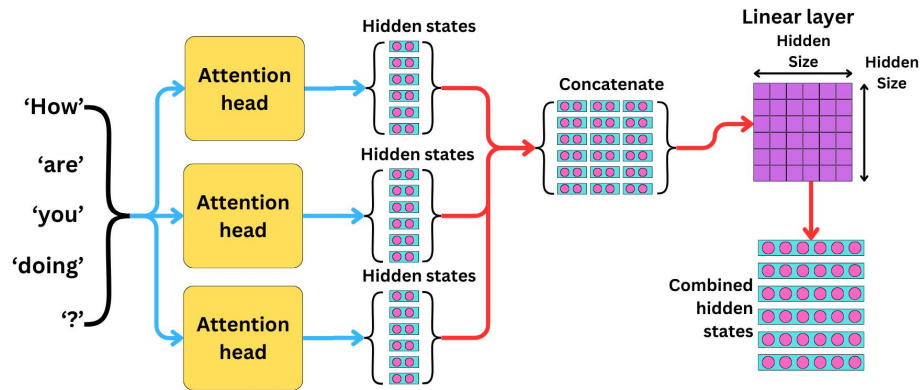
$QK^T)V$

These scores determine how much attention each word should pay to others.



Multi-Head Attention

Instead of one attention mechanism, transformers use multiple attention heads running in parallel. Each head captures different relationships or patterns in the data, enriching the model's understanding.



How to understand it the best? Implement it

"What I cannot create I do not understand"

Richard Feynman

